

Large Language Models (LLMs)

Foundations and Modern Architectures

Unit 1: Foundations of Large Language Models

A course for students, developers, and AI engineers

Chapters 1–3: Introduction to LLMs, NLP Fundamentals, and the Transformer Revolution

Contents

Introduction to LLMs	3
Unit Overview	3
Lesson 1.1 — What is a Large Language Model?	3
How does prediction become intelligence?	3
LLMs vs. traditional software	4
Lesson 1.2 — Evolution of Language Models	4
Why this history matters	4
Lesson 1.3 — Real-World Applications of LLMs	4
Chapter Summary	5
Key Takeaways	5
Glossary	6
Review Questions	6
References	6
NLP Fundamentals	7
Chapter Overview	7
Lesson 2.1 — Basics of Natural Language Processing	7
Core NLP tasks	7
Why NLP fundamentals still matter for LLMs	7
Lesson 2.2 — Text Preprocessing	7
How modern LLMs handle preprocessing differently	8
Lesson 2.3 — Tokens, Vocabulary, and Embeddings	8
Tokens	8
Vocabulary	8
Embeddings	9
From tokens to model input	9
Chapter Summary	9
Key Takeaways	9
Glossary	10
Review Questions	10
References	10
The Transformer Revolution	11
Chapter Overview	11
Lesson 3.1 — Why Transformers Changed AI	11
The consequence: scale becomes practical	11
Lesson 3.2 — Encoder vs. Decoder	11
Why most modern LLMs are decoder-only	12
Lesson 3.3 — Self-Attention (Conceptual Overview)	12
The intuition	12
The three roles: Query, Key, Value (conceptual)	13
Multi-head attention (brief preview)	13

Why this matters for everything that follows	13
Chapter Summary	13
Key Takeaways	13
Glossary	14
Review Questions	14
References	14
Unit 1 Summary	14

Introduction to LLMs

Unit Overview

Large Language Models, commonly abbreviated as LLMs, have become one of the most transformative technologies in modern computing. They power chatbots, coding assistants, search engines, and countless products used every day by millions of people. This unit builds your foundation: what LLMs are, how the field of language modeling evolved to reach this point, the basics of Natural Language Processing that LLMs are built on, and the Transformer architecture that made today’s models possible.

By the end of this unit, you will be able to explain what an LLM is in precise technical terms, trace the historical path from rule-based systems to modern Transformers, describe how raw text becomes something a model can process, and explain — at a conceptual level — why self-attention was such an important breakthrough.

Lesson 1.1 — What is a Large Language Model?

A Large Language Model is a neural network trained on enormous amounts of text to predict the next piece of text given everything that came before it. That single idea — predicting the next token — is deceptively powerful. If a model becomes extremely good at predicting what word or sub-word comes next in any piece of human-written text, it must, in the process, build an internal representation of grammar, facts about the world, reasoning patterns, and even style and tone.

Three ingredients define an LLM:

1. **Scale.** Modern LLMs contain from a few billion to over a trillion parameters — the adjustable numbers inside the network that are tuned during training. “Large” refers to both the number of parameters and the volume of training text, which can reach trillions of words drawn from books, websites, code repositories, and more.
2. **Architecture.** Nearly every modern LLM is built on the Transformer architecture, introduced in 2017. We explore this in depth in Chapter 3 and Unit 2.
3. **Generality.** Unlike older NLP systems built for one narrow task (like only translating French to English), a single LLM can summarize, translate, write code, answer questions, and hold a conversation, all using the same underlying model.

Figure 1.1: The Language Modeling Objective. A diagram showing a partial sentence (“The cat sat on the”) feeding into a neural network, which outputs a probability distribution over possible next words (e.g., “mat”: 62%, “chair”: 14%, “roof”: 4%, other words: remainder).

How does prediction become intelligence?

It helps to think of language modeling as an extremely rich training signal. To predict the next word in “The capital of France is ____” correctly, a model must implicitly learn a geographic fact. To predict the next line of a function correctly, it must learn how code logic flows. Training on trillions of such predictions, across nearly every domain of written human knowledge, produces a system with broad, general capabilities — even though the training objective itself is simple.

It is important to be precise about what this means. An LLM does not “know” facts the way a database does; it has learned statistical patterns that often — but not always — correspond to true and useful information. This is why LLMs can be

remarkably capable and also occasionally confidently wrong, a phenomenon known as **hallucination**, which we return to later in the course.

LLMs vs. traditional software

Traditional software is written as explicit instructions: if this condition, do that action. An LLM is not programmed with rules for grammar or facts. Instead, its behavior emerges from patterns learned in data. This is the essential shift from *rule-based* to *learned* systems, and it is the reason LLMs generalize so well to situations their designers never explicitly anticipated.

Lesson 1.2 — Evolution of Language Models

Language modeling did not begin with Transformers. Understanding the path that led here helps you appreciate why each architectural shift mattered.

Stage 1: Rule-based and statistical systems (1950s–2000s). Early machine translation and text systems relied on hand-written grammar rules and dictionaries. By the 1990s, **n-gram models** became dominant: these estimate the probability of a word based on the previous n words, using simple frequency counts from large text corpora. N-gram models were fast and easy to build but could not capture long-range context — a model built on trigrams (three-word windows) has no memory of something mentioned ten words earlier.

Stage 2: Neural language models (2010–2017). Researchers began using neural networks, particularly **Recurrent Neural Networks (RNNs)** and their improved variant, **Long Short-Term Memory (LSTM)** networks, to model language. These process text sequentially, one word at a time, carrying a “hidden state” forward as memory. This was a major step forward, allowing models to capture longer dependencies and produce more fluent text. However, RNNs still struggled with very long sequences (the memory tends to fade, known as the *vanishing gradient problem*) and, critically, they process text one step at a time, which makes them slow to train on the massive datasets modern models require.

Figure 1.2: Timeline of Language Model Evolution. A horizontal timeline showing four eras: *N-gram Models (1990s) → RNN/LSTM Models (2010–2016) → Transformer Models (2017 onward) → Large-Scale Pretrained LLMs (2018–present)*, with example systems labeled under each era (e.g., GPT-3, BERT, GPT-4 under the last era).

Stage 3: The Transformer era (2017 onward). In 2017, the paper “Attention Is All You Need” introduced the Transformer architecture, which replaced sequential recurrence with a mechanism called **self-attention** that processes an entire sequence in parallel while still capturing relationships between distant words. This unlocked massive parallel training on modern hardware (GPUs and TPUs), which in turn made it practical to train models on far larger datasets.

Stage 4: Large-scale pretraining and scaling (2018–present). Models such as BERT (2018), GPT-2 (2019), GPT-3 (2020), and their many successors demonstrated that simply scaling up Transformer models — more parameters, more data, more compute — produced steady and sometimes surprising gains in capability. This observation, often called the **scaling hypothesis**, has driven much of the field’s progress and investment since.

Why this history matters

Each stage solved a limitation of the one before it: n-grams lacked memory, RNNs had memory but were slow and forgetful over long distances, and Transformers gave models both long-range memory and parallel, efficient training. Recognizing this progression will help you understand *why* the Transformer’s design choices exist, not just *what* they are.

Lesson 1.3 — Real-World Applications of LLMs

LLMs are now embedded in a very wide range of products and workflows. Below are the major application categories you are likely to encounter or build.

Conversational assistants. Chat-based systems (such as customer support bots and general-purpose assistants) use LLMs to hold multi-turn conversations, answer questions, and complete tasks described in natural language.

Code generation and developer tools. LLMs trained heavily on source code can autocomplete functions, explain unfamiliar code, generate tests, and even refactor entire files, dramatically changing developer workflows.

Search and information retrieval. Modern search engines increasingly use LLMs to generate direct, synthesized answers to queries, often combined with a technique called **retrieval-augmented generation (RAG)**, where the model is given relevant documents to ground its answer.

Content creation. LLMs assist with drafting articles, marketing copy, emails, and creative writing, typically as a collaborative tool where a human edits and directs the output.

Summarization and document analysis. LLMs can condense long reports, legal documents, or research papers into concise summaries, or extract specific structured information (such as dates, names, or figures) from unstructured text.

Translation and accessibility. LLMs provide high-quality translation across many languages and can assist with accessibility tasks such as simplifying complex text.

Domain-specific copilots. Specialized LLM applications now exist for law, medicine, finance, and scientific research, typically fine-tuned or carefully prompted with domain knowledge and often paired with retrieval systems for accuracy.

Table 1.1: Application Categories and Example Tasks

Category	Example Task
Conversational AI	Customer support chatbot
Developer tools	Code completion, bug explanation
Search	Synthesized answers with citations
Content creation	Blog draft, marketing copy
Summarization	Condensing a 50-page report
Translation	Real-time multilingual chat
Domain copilots	Legal contract review assistant

A recurring theme across all these applications is that LLMs work best when paired thoughtfully with human oversight, additional tools (like search or calculators), and, where accuracy is critical, retrieval systems that ground responses in verified sources.

Chapter Summary

This chapter introduced Large Language Models as neural networks trained to predict the next token in text, a simple objective that gives rise to broad, general capabilities at sufficient scale. We traced the evolution of language modeling from rule-based systems, through statistical n-gram models and sequential RNN/LSTM networks, to the parallel, attention-based Transformer architecture that powers virtually every modern LLM. Finally, we surveyed the wide range of real-world applications LLMs now support, from conversational assistants to domain-specific copilots.

Key Takeaways

- An LLM predicts the next token in a sequence; this simple objective, applied at massive scale, yields broad general capability.
- LLM behavior emerges from learned statistical patterns in data, not hand-written rules.
- Language modeling evolved through four stages: rule-based systems, statistical n-grams, RNN/LSTM neural models, and Transformer-based models.
- The Transformer’s self-attention mechanism enabled both long-range context and efficient parallel training, which n-grams and RNNs could not offer together.
- The scaling hypothesis — that more parameters, data, and compute reliably improve capability — has driven the field since 2018.
- LLMs today power conversational assistants, code tools, search, content creation, summarization, translation, and domain-specific copilots.

Glossary

Large Language Model (LLM): A neural network trained on large volumes of text to predict the next token, giving it broad language and reasoning capabilities. **Parameter:** An internal, adjustable numeric value in a neural network, tuned during training. **N-gram model:** A statistical model that predicts a word from the preceding $n - 1$ words using frequency counts. **RNN / LSTM:** Recurrent neural network architectures that process sequences one step at a time, carrying forward a memory state. **Transformer:** A neural network architecture built around self-attention, enabling parallel processing of sequences. **Scaling hypothesis:** The observation that increasing model size, data, and compute tends to reliably improve LLM performance. **Hallucination:** When an LLM generates plausible-sounding but factually incorrect or fabricated content. **Retrieval-augmented generation (RAG):** A technique where an LLM is given retrieved documents to ground its response in verified information.

Review Questions

1. In one sentence, define the core training objective of a Large Language Model.
2. What three factors together define what makes a model an “LLM”?
3. Why couldn’t n-gram models capture long-range context in text?
4. What key limitation of RNNs did the Transformer architecture solve?
5. What is the “scaling hypothesis,” and why has it been influential?
6. Give two examples of real-world LLM applications and briefly describe each.
7. Explain, in your own words, why LLM behavior is described as “emergent” rather than explicitly programmed.
8. What is hallucination, and why does it occur in LLMs?
9. What role does retrieval-augmented generation play in improving LLM accuracy?
10. Why is understanding the historical evolution of language models useful when studying Transformers?

References

1. Vaswani, A., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems.
2. Jurafsky, D., & Martin, J. H. *Speech and Language Processing* (3rd ed. draft). Stanford University.
3. Brown, T., et al. (2020). *Language Models are Few-Shot Learners*. NeurIPS.
4. Radford, A., et al. (2019). *Language Models are Unsupervised Multitask Learners*. OpenAI.
5. Devlin, J., et al. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv.

NLP Fundamentals

Chapter Overview

Before an LLM can predict anything, raw text must be transformed into a form a neural network can process: numbers. This chapter covers the foundational Natural Language Processing (NLP) concepts every LLM practitioner needs — what NLP is, how text is cleaned and prepared, and how text becomes tokens, vocabularies, and ultimately the embeddings that feed into a model.

Lesson 2.1 — Basics of Natural Language Processing

Natural Language Processing (NLP) is the field of computer science concerned with enabling computers to process, understand, and generate human language. NLP predates LLMs by decades and includes many sub-tasks that remain relevant building blocks today.

Core NLP tasks

- **Tokenization:** splitting text into smaller units (words, sub-words, or characters).
- **Part-of-speech (POS) tagging:** labeling each word with its grammatical role (noun, verb, adjective, etc.).
- **Named entity recognition (NER):** identifying names of people, organizations, locations, and dates within text.
- **Parsing:** analyzing the grammatical structure of a sentence.
- **Sentiment analysis:** determining the emotional tone of a piece of text.
- **Machine translation:** converting text from one language to another.
- **Text classification:** assigning a category label to a document (e.g., spam vs. not spam).

Figure 2.1: The NLP Pipeline. A flow diagram showing raw text moving through sequential stages: *Raw Text* → *Tokenization* → *Preprocessing* → *Feature Representation (Embeddings)* → *Model* → *Output (classification, generation, etc.)*.

Why NLP fundamentals still matter for LLMs

Modern LLMs handle many of these tasks implicitly, through the single next-token prediction objective, rather than through separate specialized systems. However, the underlying concepts — tokenization, vocabulary, and text representation — remain essential building blocks. Understanding them is necessary to understand how LLMs are built, trained, and sometimes why they fail (for example, an LLM’s difficulty with certain arithmetic often traces back to how numbers are tokenized).

Lesson 2.2 — Text Preprocessing

Before deep learning became dominant, preprocessing text heavily (removing punctuation, lowercasing, removing “stop words” like “the” and “is”, stemming words to their root form) was standard practice, because older statistical models benefited from a smaller, cleaner vocabulary.

Common preprocessing steps:

1. **Lowercasing:** converting all text to lowercase so “Apple” and “apple” are treated identically (used selectively today, since case can carry meaning, e.g., distinguishing “US” the country from “us” the pronoun).
2. **Removing punctuation and special characters:** stripping symbols not relevant to the task.
3. **Stop word removal:** removing very common words that carry little distinguishing meaning in tasks like search or classification.
4. **Stemming:** crudely truncating words to an approximate root (e.g., “running” → “run”) using rule-based suffix stripping.
5. **Lemmatization:** a more linguistically accurate version of stemming that maps words to their true dictionary root form (e.g., “better” → “good”) using vocabulary and grammar knowledge.
6. **Normalization:** standardizing text formats, such as converting numbers, dates, or unicode characters into a consistent form.

How modern LLMs handle preprocessing differently

Modern LLMs generally use **minimal preprocessing**. Rather than aggressively cleaning text, they instead rely on a robust tokenization scheme (covered in the next lesson) that can represent almost any raw text, including punctuation, casing, emojis, and code syntax, directly. This is because deep learning models are capable of learning which details matter from data itself, and removing information (like casing or punctuation) can throw away useful signal. Preprocessing is still important for tasks like data cleaning at the dataset level — removing duplicate documents, filtering low-quality or toxic content, and de-identifying private information before training — even if per-word cleaning steps like stemming are rarely used today.

Table 2.1: Classic Preprocessing vs. Modern LLM Practice

Aspect	Classic NLP Pipelines	Modern LLM Practice
Casing	Often lowercased	Usually preserved
Punctuation	Often removed	Usually preserved
Stop words	Often removed	Usually preserved
Stemming/Lemmatization	Common	Rarely used
Main cleaning focus	Per-word normalization	Dataset-level filtering and deduplication

Lesson 2.3 — Tokens, Vocabulary, and Embeddings

Tokens

A **token** is the basic unit of text that a model processes. Early NLP systems often used whole words as tokens, but this creates a problem: any word not seen during training (an “out-of-vocabulary” word) cannot be represented. Modern LLMs solve this with **subword tokenization**, which breaks words into smaller, reusable pieces. For example, the word “unbelievable” might be split into tokens like “un”, “believ”, and “able”. This allows the model to represent any word, even unfamiliar or made-up ones, by combining known sub-word pieces.

The most widely used subword tokenization algorithm is **Byte Pair Encoding (BPE)** and its variants. BPE works by starting with individual characters and iteratively merging the most frequently occurring adjacent pairs into new tokens, building up a fixed-size vocabulary of the most useful sub-word units from the training data.

Figure 2.2: Subword Tokenization Example. A diagram showing the word “tokenization” being split into sub-word pieces such as “token” + “ization”, each mapped to a unique numeric token ID (e.g., “token” → 14801, “ization” → 3927).

Vocabulary

A model’s **vocabulary** is the fixed set of all tokens it knows how to use, typically ranging from about 30,000 to over 100,000 entries for modern LLMs. Every piece of text the model reads or generates must be expressed using only tokens from this fixed set. Choosing vocabulary size involves a trade-off: a larger vocabulary lets the model represent common words in

fewer tokens (more efficient), but increases the size of the model's output layer and the amount of training data needed to learn rare tokens well.

Embeddings

Once text is split into tokens, each token is converted into a numeric ID (its position in the vocabulary), and then that ID is mapped to an **embedding**: a vector of real numbers (typically several hundred to a few thousand dimensions) that represents the token's meaning in a continuous numeric space. These embeddings are not designed by hand; they are learned during training.

The key property of a well-trained embedding space is that **similar meanings end up close together**. Words like “king” and “queen” will have embedding vectors that are near each other in this high-dimensional space, while unrelated words like “king” and “bicycle” will be far apart. Famously, embedding spaces also tend to capture relationships as directions: the vector difference between “king” and “queen” is often similar to the difference between “man” and “woman”, reflecting a shared “gender” relationship learned purely from data.

Figure 2.3: A Simplified 2D Embedding Space. *A scatter plot showing word points positioned by meaning, with clusters such as {king, queen, prince} near each other, {apple, banana, orange} in a separate cluster, and {run, walk, jump} in another, illustrating that semantically related words are positioned close together.*

From tokens to model input

To summarize the full path from raw text to something a Transformer can process:

1. **Raw text** — “The cat sat down.”
2. **Tokenization** — split into subword tokens, e.g., [“The”, “cat”, “sat”, “down”, “.”]
3. **Token IDs** — each token mapped to an integer via the vocabulary, e.g., [464, 3797, 3332, 866, 13]
4. **Embeddings** — each ID mapped to a learned vector, e.g., a 768-dimensional vector per token
5. **Model input** — the sequence of embeddings, plus positional information (covered in Chapter 3), is fed into the Transformer

Chapter Summary

This chapter covered the NLP building blocks that underlie every LLM. We reviewed core NLP tasks such as tokenization, POS tagging, and named entity recognition, and contrasted classic heavy preprocessing (lowercasing, stop-word removal, stemming) with the minimal, dataset-level preprocessing typical of modern LLM pipelines. We then examined how subword tokenization (like BPE) solves the out-of-vocabulary problem, how a fixed vocabulary constrains what a model can represent, and how embeddings convert discrete tokens into continuous numeric vectors that capture semantic meaning.

Key Takeaways

- NLP is the broader field of enabling computers to process human language; LLMs build on, and often subsume, classic NLP tasks.
- Modern LLMs use minimal per-word preprocessing, preferring to preserve information like casing and punctuation, while focusing cleaning efforts at the dataset level.
- Subword tokenization (e.g., Byte Pair Encoding) solves the out-of-vocabulary problem by breaking words into reusable pieces.
- A model's vocabulary is a fixed set of tokens; vocabulary size trades off efficiency against model and data requirements.
- Embeddings map tokens to numeric vectors such that semantically similar tokens are positioned close together in vector space.
- The pipeline from raw text to model input is: text → tokens → token IDs → embeddings → model input.

Glossary

Token: The basic text unit (word, sub-word, or character) that a model processes. **Subword tokenization:** A tokenization method that splits words into smaller, reusable pieces to handle unseen words. **Byte Pair Encoding (BPE):** A subword tokenization algorithm that iteratively merges frequent adjacent character pairs. **Vocabulary:** The fixed set of all tokens a model can recognize and generate. **Out-of-vocabulary (OOV):** A word or piece of text that cannot be represented because it is not in the model's vocabulary. **Embedding:** A learned numeric vector representation of a token that captures aspects of its meaning. **Stemming:** Rule-based truncation of a word to an approximate root form. **Lemmatization:** Mapping a word to its true dictionary root form using linguistic knowledge.

Review Questions

1. What is the difference between a word and a token in the context of NLP?
2. Why do modern LLMs prefer subword tokenization over whole-word tokenization?
3. Describe, in your own words, how Byte Pair Encoding builds its vocabulary.
4. What is the out-of-vocabulary problem, and how does subword tokenization solve it?
5. Name three classic preprocessing steps and explain why modern LLMs largely avoid them.
6. What is an embedding, and what property should a well-trained embedding space have?
7. Give an example of a semantic relationship that can be captured as a direction in embedding space.
8. What is the trade-off involved in choosing a vocabulary size?
9. List, in order, the five steps that convert raw text into model input.
10. Why might an LLM struggle with certain arithmetic tasks, based on how tokenization works?

References

1. Sennrich, R., Haddow, B., & Birch, A. (2016). *Neural Machine Translation of Rare Words with Subword Units*. ACL.
2. Mikolov, T., et al. (2013). *Efficient Estimation of Word Representations in Vector Space*. arXiv.
3. Jurafsky, D., & Martin, J. H. *Speech and Language Processing* (3rd ed. draft), Chapters 2–6. Stanford University.
4. Kudo, T., & Richardson, J. (2018). *SentencePiece: A Simple and Language Independent Subword Tokenizer*. EMNLP.

The Transformer Revolution

Chapter Overview

This chapter introduces the architecture that made modern LLMs possible: the Transformer. We look at why Transformers represented a genuine breakthrough over prior architectures, the difference between encoder and decoder components, and — at a conceptual level, without requiring linear algebra — how self-attention actually works. A deeper, fully technical treatment of the architecture follows in Unit 2.

Lesson 3.1 — Why Transformers Changed AI

Recall from Chapter 1 that RNNs process text sequentially: to understand word 50 in a sentence, the model must first process words 1 through 49, carrying information forward step by step. This creates two serious problems.

Problem 1: Long-range memory decay. As information passes through many sequential steps, earlier details tend to fade — much like a message garbled after being whispered through a long chain of people. This made it difficult for RNNs to relate the beginning of a long passage to its end.

Problem 2: No parallelization. Because each step depends on the previous one’s output, RNNs cannot be parallelized across the sequence dimension. Training must proceed one time-step at a time, which is slow and makes it impractical to fully exploit modern GPU hardware, which excels at doing many computations simultaneously.

The Transformer, introduced in the 2017 paper “Attention Is All You Need,” solved both problems with a single mechanism: **self-attention**. Self-attention allows every token in a sequence to directly look at, and weigh the relevance of, every other token in the sequence — regardless of distance — in a single computational step. There is no step-by-step chain to propagate information through, so there is no long-range decay problem. And because every token’s attention calculation can be computed simultaneously, the whole operation parallelizes efficiently on GPUs.

Figure 3.1: RNN vs. Transformer Information Flow. *Two side-by-side diagrams. Left: an RNN, shown as a chain of boxes each passing a hidden state to the next, illustrating sequential, one-at-a-time processing. Right: a Transformer, shown as a set of tokens all connected to each other directly via attention lines, illustrating parallel, all-at-once processing.*

The consequence: scale becomes practical

Because Transformers train efficiently in parallel, researchers could feasibly train much larger models on much larger datasets within reasonable time frames. This is the direct architectural reason the scaling hypothesis (Chapter 1) became testable and, ultimately, so successful: the Transformer wasn’t just a better architecture in isolation, it was an architecture uniquely suited to the hardware and data scale of the following decade.

Lesson 3.2 — Encoder vs. Decoder

The original Transformer architecture actually consists of two distinct components: an **encoder** and a **decoder**. Understanding their different roles clarifies why today’s LLMs are built the way they are.

The Encoder. The encoder reads an entire input sequence at once and produces a rich contextual representation of it, where each token’s representation has been informed by every other token in the input (both before and after it — this is called *bidirectional* context). Encoders are well suited to tasks that require deeply *understanding* input text, such as classification, extracting named entities, or producing embeddings for search. BERT (2018) is a well-known encoder-only model.

The Decoder. The decoder generates text one token at a time, and — critically — each new token can only attend to tokens that came *before* it (this is called *causal* or *autoregressive* attention, enforced with a technique called masking). This restriction exists because, at generation time, future tokens do not exist yet; a decoder trained without this restriction would be able to “cheat” by looking ahead during training. Decoders are well suited to *generation* tasks: writing, completing, and conversing. GPT-style models are decoder-only.

The Encoder-Decoder combination. The original Transformer combined both: an encoder to process an input (such as a source-language sentence), and a decoder that generates an output (such as the translated sentence) while also attending back to the encoder’s representation of the input. This design remains popular for tasks with a clear input-to-output mapping, such as translation and summarization. T5 is a well-known encoder-decoder model.

Table 3.1: Encoder vs. Decoder vs. Encoder-Decoder

Property	Encoder-only	Decoder-only	Encoder-Decoder
Context direction	Bidirectional	Causal (left-to-right)	Both
Best suited for	Understanding, classification	Open-ended generation	Input-to-output mapping
Example model	BERT	GPT family	T5
Typical use case	Search embeddings, NER	Chat, code generation	Translation, summarization

Why most modern LLMs are decoder-only

Most of today’s best-known general-purpose LLMs (the GPT family and many others) are **decoder-only**. This design won out for general-purpose assistants primarily because a single, simple, causal next-token objective can be applied uniformly to nearly any task, framed purely as text generation — including tasks that once required a separate encoder-decoder setup, such as translation or summarization, simply by prompting the decoder appropriately. This unification simplified training pipelines and made it far easier to scale one architecture across a huge diversity of tasks.

Lesson 3.3 — Self-Attention (Conceptual Overview)

This lesson explains self-attention conceptually. A full mathematical treatment, including Query, Key, and Value matrices, appears in Unit 2, Chapter 1.

The intuition

Consider the sentence: “The trophy didn’t fit in the suitcase because **it** was too big.” To understand what “it” refers to, a reader must look back and weigh the relevance of every earlier word. Is “it” more related to “trophy” or “suitcase”? Human readers use context to decide “it” almost certainly means “trophy” here (because a suitcase being too big wouldn’t prevent a trophy from fitting inside it).

Self-attention gives a model a mechanism to do exactly this, automatically and for every token simultaneously. For each token, self-attention computes an **attention score** against every other token in the sequence, representing how relevant that other token is for interpreting the current one. These scores are then used as weights to blend information from across the whole sequence into a new, context-aware representation of the current token.

Figure 3.2: Self-Attention Weights Example. A diagram of the sentence “The trophy didn’t fit in the suitcase because it was too big,” with the word “it” highlighted, and arrows of varying thickness connecting “it” back to every other word, where the arrow to “trophy” is thick (high attention weight) and the arrow to “suitcase” is thin (low attention weight), visually representing which words the model is “paying attention to” when interpreting “it.”

The three roles: Query, Key, Value (conceptual)

Without diving into the underlying matrix mathematics, it helps to know the three roles every token plays simultaneously in self-attention:

- **Query:** represents what the current token is “looking for” in the rest of the sequence.
- **Key:** represents what each token “offers” as a label for others to match against.
- **Value:** represents the actual content each token contributes once it has been deemed relevant.

Loosely: every token asks a question (Query), every token advertises what it contains (Key), the model matches questions to advertisements to compute relevance scores, and then blends together the actual content (Value) of the most relevant tokens, weighted by those scores. This process happens for every token, with respect to every other token, at the same time — which is precisely why it parallelizes so well.

Multi-head attention (brief preview)

In practice, Transformers don’t compute just one set of attention scores per token; they compute several in parallel, called **attention heads**, each potentially learning to focus on different types of relationships — one head might specialize in tracking grammatical subject-verb agreement, another in resolving pronouns like “it” above, and another in something less human-interpretable. Their results are combined afterward. This technique, called **multi-head attention**, is covered in full technical detail in Unit 2.

Why this matters for everything that follows

Self-attention is the mechanism that gives Transformers their defining strength: the ability to flexibly and dynamically weigh context from anywhere in a sequence, without the memory decay of RNNs and without sacrificing parallel training. Every concept in the remainder of this course — from positional encoding to multi-head attention to the training procedures used to build GPT-scale models — builds directly on this idea.

Chapter Summary

This chapter explained why the Transformer architecture was a genuine breakthrough: self-attention solved both the long-range memory decay and the lack of parallelization that limited RNNs. We distinguished encoder architectures (bidirectional, suited to understanding tasks), decoder architectures (causal, suited to generation), and encoder-decoder combinations (suited to input-to-output mapping tasks), and noted that most modern general-purpose LLMs are decoder-only. Finally, we built a conceptual, non-mathematical understanding of self-attention using the Query, Key, and Value framing, setting up the detailed architectural treatment in Unit 2.

Key Takeaways

- Self-attention lets every token directly relate to every other token in one step, avoiding RNNs’ long-range memory decay.
- Because attention computations for all tokens can run simultaneously, Transformers parallelize efficiently on modern hardware.
- Encoders build bidirectional representations suited to understanding tasks; decoders generate text causally, suited to open-ended generation.
- Most modern general-purpose LLMs are decoder-only, since a unified next-token objective scales well across many tasks.
- Self-attention can be understood conceptually through Query (what a token seeks), Key (what a token offers), and Value (the content a token contributes).
- Multi-head attention runs several attention computations in parallel, potentially capturing different types of relationships.

Glossary

Self-attention: A mechanism where every token computes relevance scores against every other token in a sequence to build context-aware representations. **Encoder:** A Transformer component that builds bidirectional representations of an input sequence. **Decoder:** A Transformer component that generates text causally, attending only to prior tokens. **Causal (autoregressive) attention:** An attention pattern restricted to only look at earlier tokens in a sequence. **Query, Key, Value:** The three learned roles each token plays in computing self-attention. **Multi-head attention:** Running several self-attention computations in parallel, each potentially capturing different relationships. **Attention score:** A computed value representing how relevant one token is to another.

Review Questions

1. What two specific problems of RNNs does self-attention solve?
2. Why does removing sequential dependency make Transformer training faster on GPUs?
3. Describe the difference in attention direction between an encoder and a decoder.
4. Why must a decoder use causal masking during training?
5. Give one example each of an encoder-only, decoder-only, and encoder-decoder model.
6. Why have decoder-only architectures become dominant for general-purpose LLMs?
7. In the Query/Key/Value framing, what role does each of the three represent?
8. Using the “trophy/suitcase” example, explain in your own words how self-attention resolves what “it” refers to.
9. What is multi-head attention, and why might multiple heads be useful?
10. How does self-attention connect back to the scaling hypothesis discussed in Chapter 1?

References

1. Vaswani, A., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems.
2. Devlin, J., et al. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv.
3. Raffel, C., et al. (2019). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (T5)*. arXiv.
4. Alammam, J. *The Illustrated Transformer*. jalammar.github.io.

Unit 1 Summary

Unit 1 established the foundation for everything that follows in this course. You learned that an LLM is, at its core, a neural network trained to predict the next token in text — a simple objective whose scale gives rise to broad general capability. You traced the evolutionary path from rule-based systems and n-gram models, through RNNs and LSTMs, to the Transformer architecture that defines the current era. You learned the essential NLP building blocks — tokenization, vocabulary, and embeddings — that convert raw text into a numeric form models can process. Finally, you built both an architectural and conceptual understanding of the Transformer, distinguishing encoders from decoders and understanding, at a high level, how self-attention allows models to weigh context from anywhere in a sequence. With this foundation in place, Unit 2 turns to a deeper technical treatment of Transformer architecture, training procedures, and how to effectively use LLMs in practice.